Academic Audit Parsing

Engineering Data Extraction for Student Audits

Safiullah Rattar

Carleton Smart Course Scheduler

System Context

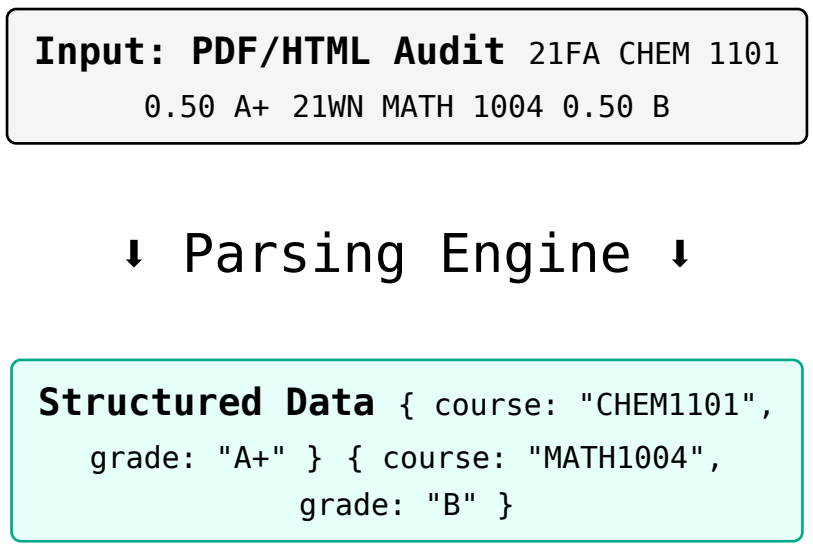
Leveraging existing student data for automation

The Engineering Gap Building a scheduler requires three key inputs:

1. **Course Data** (Prerequisites, availability)
2. **Student History** (Completed courses, grades)
3. **Optimization Algorithm** (Backtracking)

Carleton provides the data in the academic audit, but it is locked in unstructured formats (PDF/HTML). There is no public API.

Design Goal To automate course selection, we must extract structured data from these documents with **100% accuracy**. A single missed



grade could recommend a course the student is not eligible for.

Data Extraction Strategy

Evaluating extraction methodologies

Option A: OCR (Tesseract/Vision API)

- **Mechanism:** Image analysis to character prediction.
- **Failure Mode:** Probabilistic errors.
  - `B` (Grade) vs `8` (Number)
  - `I` (Incomplete) vs `1` (Digit)
- **Verdict:** Unacceptable for grade data where accuracy is critical.

Option B: Text Stream (pdftotext)

- **Mechanism:** Extract text encoding directly from PDF binary.
- **Pros:** Deterministic, fast, exact character representation.
- **Cons:** PDF text streams often lack structure (newlines, spaces).
- **Verdict:** Selected approach, but required layout handling.

The Layout Problem

When “Reading Order” breaks data logic

The raw PDF text stream is often stored “Column-Major”.

`pdftotext` reads all of Column 1 (Terms) first, then jumps back up to read Column 2 (Courses). This separates related data by dozens of lines.

pdftotext raw (Column-Major)

```
Term 22FA 22FA 21SU
Course COMP 2804 ELEC 2501 MATH 1005
Credits 0.50 0.50
```

The Fatal Flaw: “22FA” is line 2. “COMP 2804” is line 6. There is no line-by-line connection.

pdftotext -layout (Row-Major)

```
Term Course Credits Grade 22FA COMP 2804 0.50 A+ 22FA ELEC 2501 0.50
A- 21SU MATH 1005 0.50 TR
```

The Fix: Visual layout is preserved. Regex can match the entire row: `^22FA.*COMP 2804`

Regex Pattern Implementation

Extracting typed entities from columnar text

We developed a strict regex pattern to capture the specific layout of the Carleton audit.

```
/^\s*(\d{2}(?:FA|WN|SU))\s+([A-Z]{3,4})\s+(\d{4})\s+([\d.]+)\s+([A-Z+\-]+)/gm
```

Capturing Groups

1. **Term:** `21FA` (Digit + Season)
2. **Dept:** `CHEM` (3-4 Caps)
3. **Code:** `1101` (4 Digits)
4. **Credits:** `0.50` (Float)
5. **Grade:** `A+` (Alpha + modifier)

Implementation Detail The pattern enforces type constraints (e.g., Course Code must be 4 digits) to implicitly filter header text or malformed lines.

Edge Case: Repeated Courses

Handling academic history logic

Problem: A student may take a course multiple times (e.g., Fail, then Pass).
Requirement: We must only track valid completions and ignore failures/withdrawals for prerequisite checks.

```
// cli/audit-parser.ts logic
const seenCompleted = new Set<string>()

for (const match of matches) {
  // Filter out non-passing grades
  if (['F', 'X', 'UNS', 'NR', 'WD'].includes(grade)) {
     continue
  }

  // Only add if not already seen (First pass finds most recent)
  // PDF lists courses chronologically, we might need recent attempt logic
  if (!seenCompleted.has(courseId)) {
     completedCourses.push({ courseId, grade })
     seenCompleted.add(courseId)
  }
}
```

Logic: We maintain a `Set` of completed IDs. We strictly filter out failure codes (`F`, `UNS`, etc.) to ensure the scheduler does not assume a prerequisite is met when it isn’t.

Handling “Current” Courses

Differentiating In-Progress vs. Completed

Problem: Courses in progress (`CUR`) satisfy “In Progress” prerequisites (like for next term’s registration) but do not have grades.

```
const isCurrent = fullMatch.includes('CUR') || grade === 'CUR'

if (isCurrent && !seenCurrent.has(courseId)) {
  // Track separately for "In Progress" logic
  currentCourses.push({ courseId, term })
  seenCurrent.add(courseId)
} else if (isValidGrade(grade)) {
  // Track as fully completed
  completedCourses.push({ courseId, grade })
}
```

This separation is crucial. `completedCourses` allows access to courses requiring a grade (e.g., “C- in SYSC 3303”). `currentCourses` allows access to courses just requiring the credit.

Parsing Logical Requirements

Future-proofing for elective recommendations

Beyond simple course lists, we extract degree requirements to support future elective suggestions.

Requirement Pattern

```
/d+)s+([d.]+)s+credits?s+ins+ ([A-Z]{3,4}[^]+?)/
```

Captures: “2) 0.5 credit in Basic Science Electives”

Mapping Logic

```
if (title.includes('Basic Science')) {
   // Maps abstract requirement to concrete list
   category = 'SCIENCE_ELECTIVE'
}
// Result: ["BIOL1103", "CHEM1001"...]
```

Impact: This allows the system to recommend specific courses (e.g., `BIOL 1103`) when a user has an unfilled “Science Elective” slot, rather than just showing a generic placeholder.